```
    {                                                                    ❶
        _fixture = fixture;                                              ❶
    }                                                                    ❶

    [Fact]
    public async Task ConvertReturnsExpectedValue()
    {
        var customFactory = _fixture.WithWebHostBuilder(                 ❷
            (IWebHostBuilder hostBuilder) =>                             ❷
        {
            hostBuilder.ConfigureTestServices(services =>                ❸
            {
                services.RemoveAll<ICurrencyConverter>();                ❹
                services.AddScoped
                    <ICurrencyConverter, StubCurrencyConverter>();       ❺
            });
        });

        HttpClient client = customFactory.CreateClient();                ❻

        var response = await client.GetAsync("/api/currency");           ❼

        response.EnsureSuccessStatusCode();                              ❼
        var content = await response.Content.ReadAsStringAsync();        ❼

        Assert.Equal("3", content);                                      ❽
    }
}
```

❶ Implements the required interface and injects it into the constructor

❷ Creates a custom factory with the additional configuration

❸ ConfigureTestServices executes after all other DI services are configured in your real app.

❹ Removes all implementations of ICurrency-Converter from the DI container

❺ Adds the test service as a replacement

❻ Calling CreateClient bootstraps the application and starts the TestServer.

❼ Invokes the currency converter endpoint

❽ As the test converter always returns 3, so does the API endpoint.

There are a couple of important points to note in this example: